Understanding Neural Network Training Dynamics

Session 11: Hyperparameter Exploration with Fashion-MNIST

Luisa Johanna Kaczmarek · Student ID 16242
Emerging Topics in Data Analytics & Management · IE University

1. Methodology

All experiments were conducted on Fashion-MNIST (50,000 training / 10,000 validation / 10,000 test samples, 28×28 grayscale, 10 classes), normalised to [0, 1]. A baseline — Flatten → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax), Adam lr=0.001, batch size 64, 30 epochs — served as the reference for all comparisons. Fair comparison was enforced by the one-variable-at-a-time principle. The test set was evaluated exactly once on the final configuration.

2. Key Findings

2.1 Baseline ANN

The baseline showed clear overfitting: val loss bottomed at epoch 6 (0.336) then rose to 0.474 by epoch 30, while training loss fell to 0.128. Val accuracy plateaued at 0.884, giving a 6.9pp train-val accuracy gap.

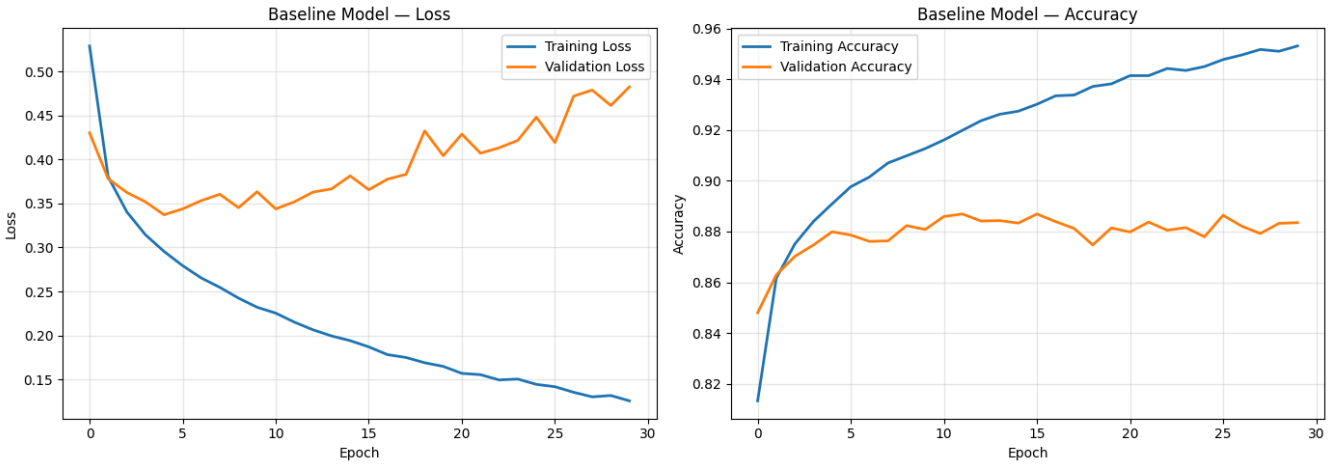

Figure 1. Baseline ANN: training and validation loss/accuracy over 30 epochs.

2.2 Experiment A — Learning Rate

lr=0.1 failed completely (val acc 0.096, val loss 2.31 — random-guess level). lr=0.01 converged faster and achieved lower final val loss (0.4418) than lr=0.001 (0.5068). Most surprisingly, lr=0.0001 achieved the best final val accuracy (0.8805) and lowest val loss (0.3331) of all five rates — challenging the assumption that very small LRs are impractical within a 30-epoch budget.

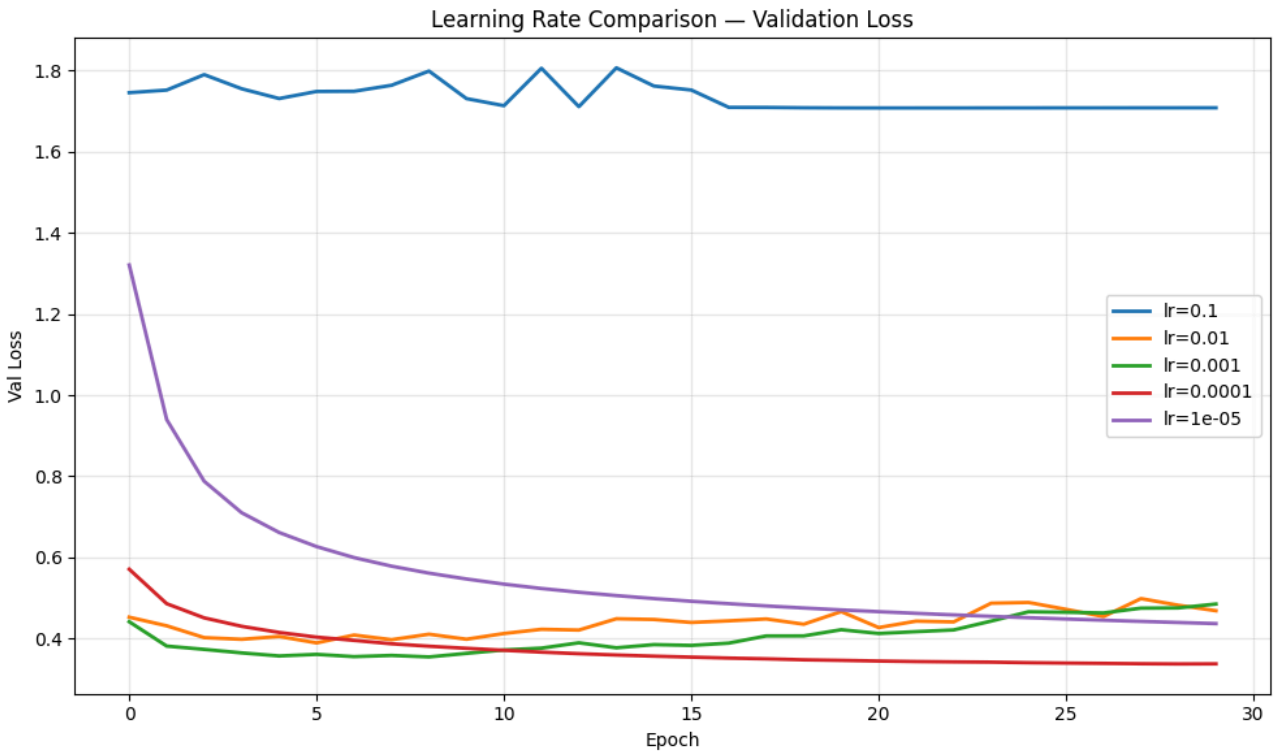

Figure 2a. Validation loss overlay for all five learning rates.

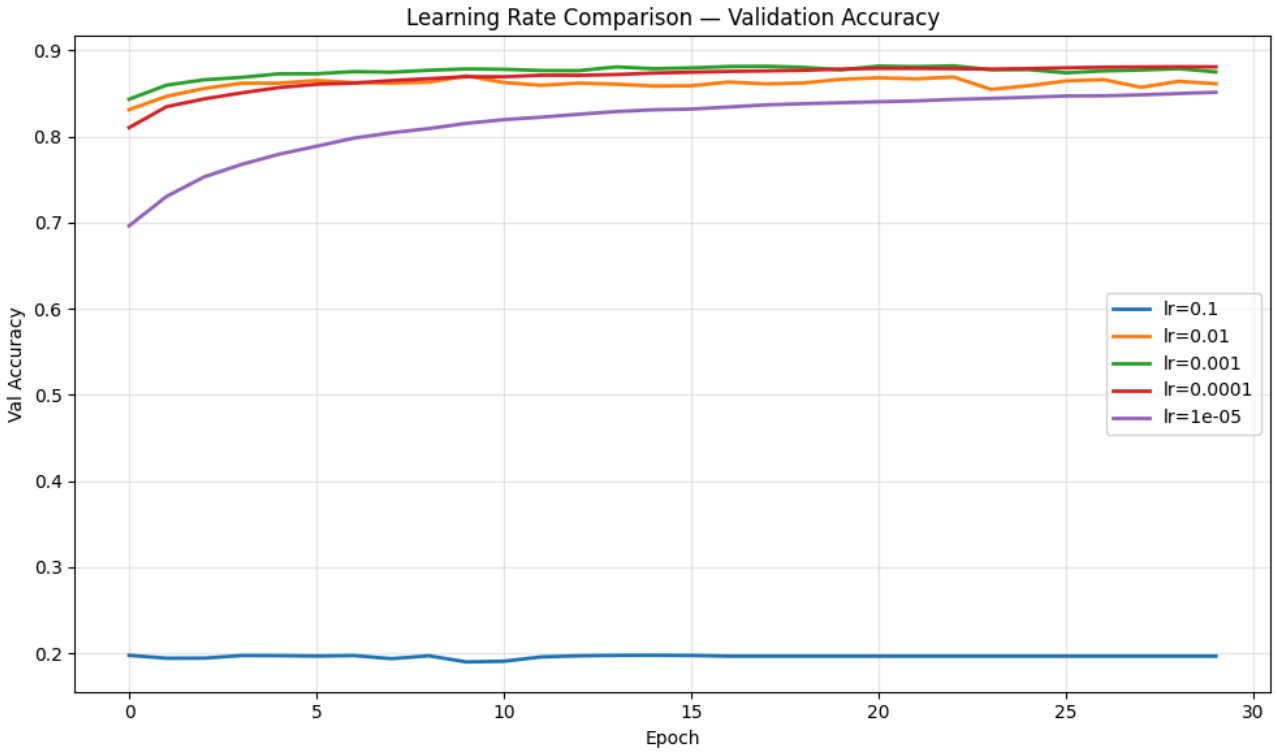

Figure 2b. Validation accuracy overlay for all five learning rates.

LR	Train Acc	Val Acc	Val Loss (ep30)	Best Val Ep
0.1	0.1014	0.0955	2.3094	1
0.01	0.8997	0.8700	0.4418	6
0.001 ★	0.9520	0.8754	0.5068	7
0.0001	0.9130	0.8805	0.3331	29
0.00001	0.8572	0.8491	0.4299	30

2.3 Experiment B — Batch Size

Large batches produced smoother curves and lower final val loss: bs=1024 achieved val loss 0.3573 at only 0.45 s/epoch vs bs=16 at 0.5295 and 8.55 s/epoch. Val accuracy was similar across all sizes (0.875–0.882), suggesting the gradient-noise regularisation benefit of small batches did not materialise within 30 epochs. The speed–quality trade-off strongly favoured bs=256–1024.

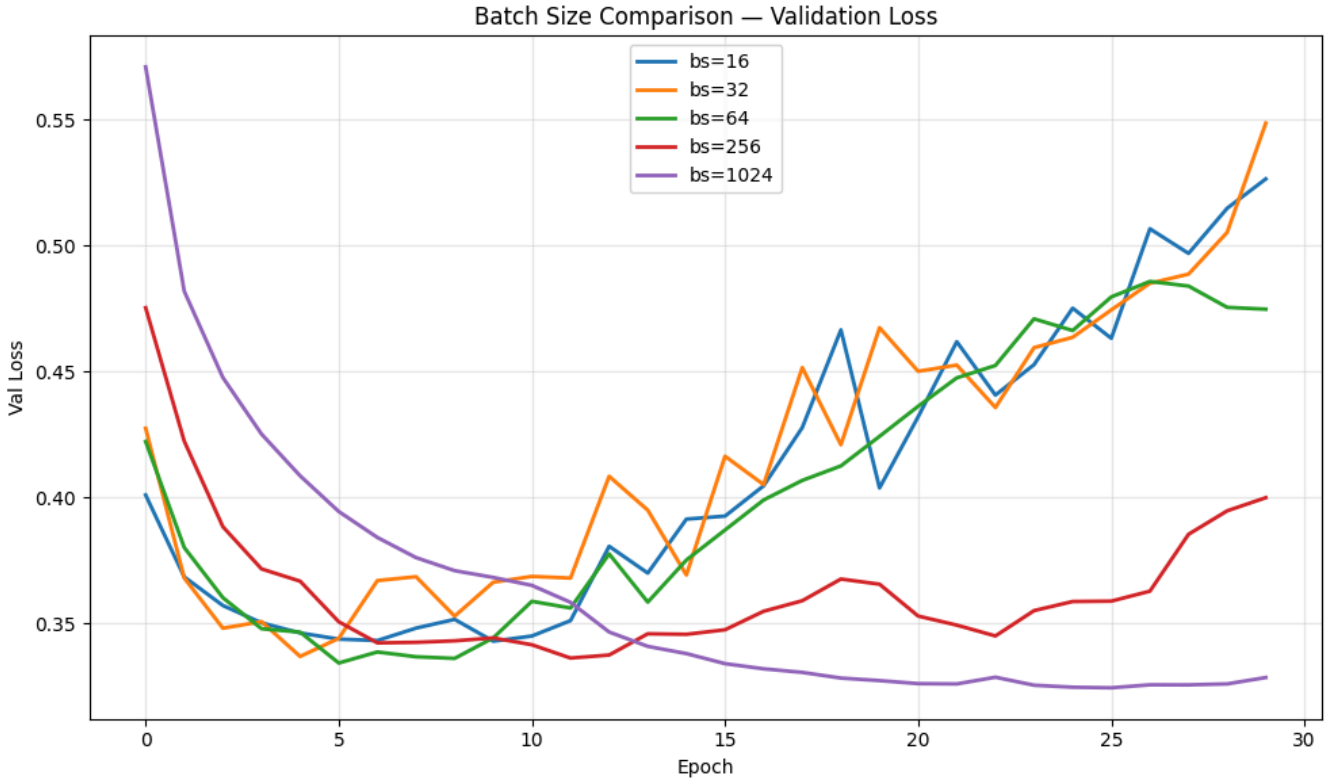

Figure 3a. Validation loss overlay for all five batch sizes.

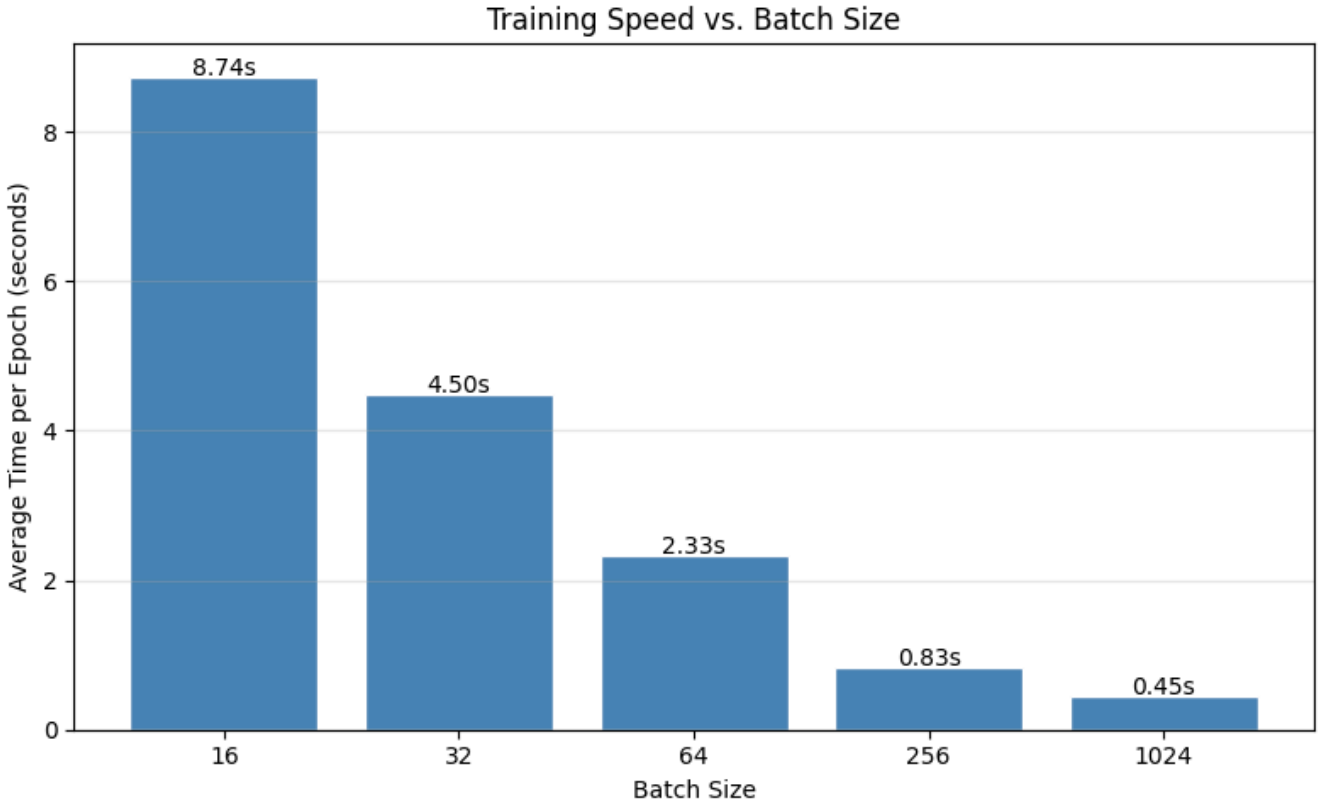

Figure 3b. Average training time per epoch by batch size.

Batch Size	Train Acc	Val Acc	Val Loss	Time/Epoch (s)
16	0.9474	0.8799	0.5295	8.55
32	0.9523	0.8820	0.5173	4.36
64 ★	0.9536	0.8769	0.4934	2.27
256	0.9406	0.8775	0.3924	0.80
1024	0.9211	0.8750	0.3573	0.45

2.4 Experiment C — Optimizer

SGD+Momentum produced the lowest final val loss (0.3520), outperforming Adam (0.4671) despite Adam converging faster. This supports the sharp-minima hypothesis: Adam's adaptive updates find narrower minima, while SGD+Momentum's slower trajectory finds flatter, better-generalising ones. RMSprop matched Adam's early speed but diverged more steeply after its best val epoch (9), ending at 0.5537.

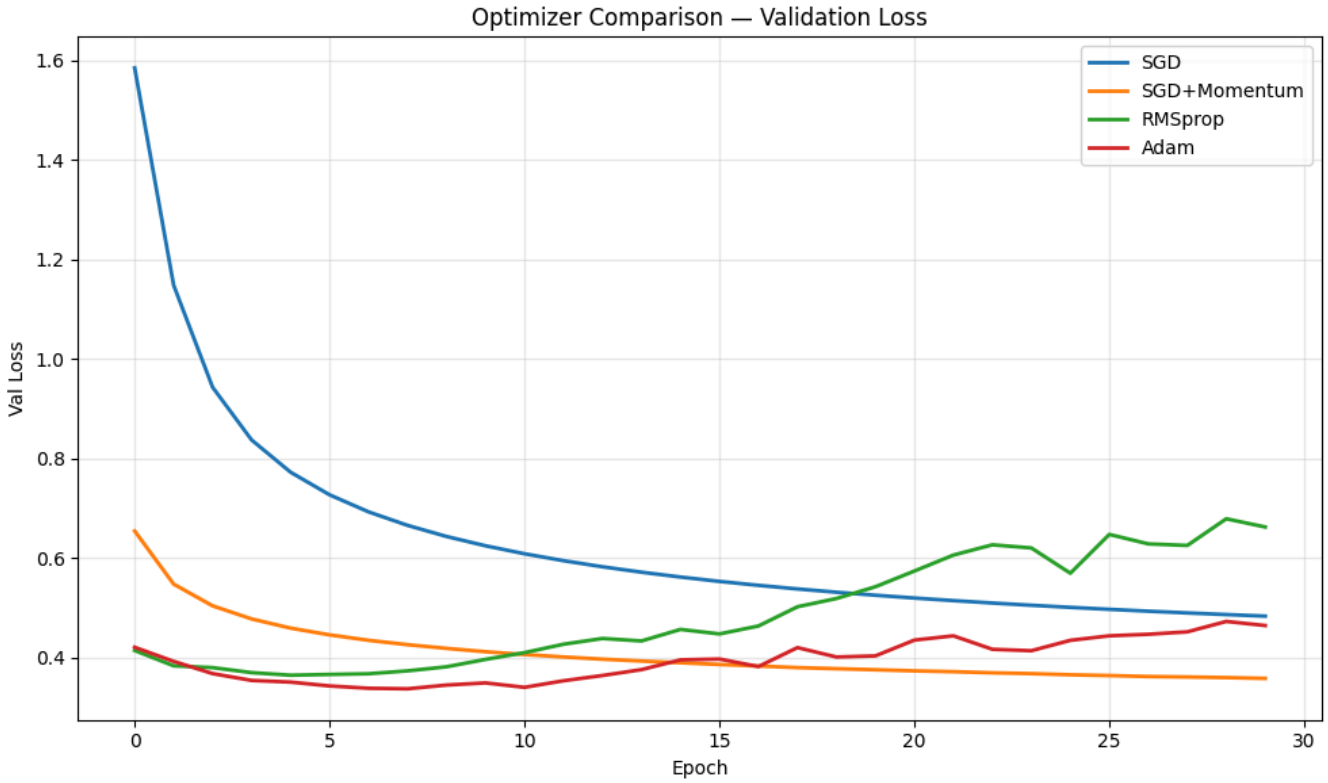

Figure 4a. Validation loss overlay for all four optimizers.

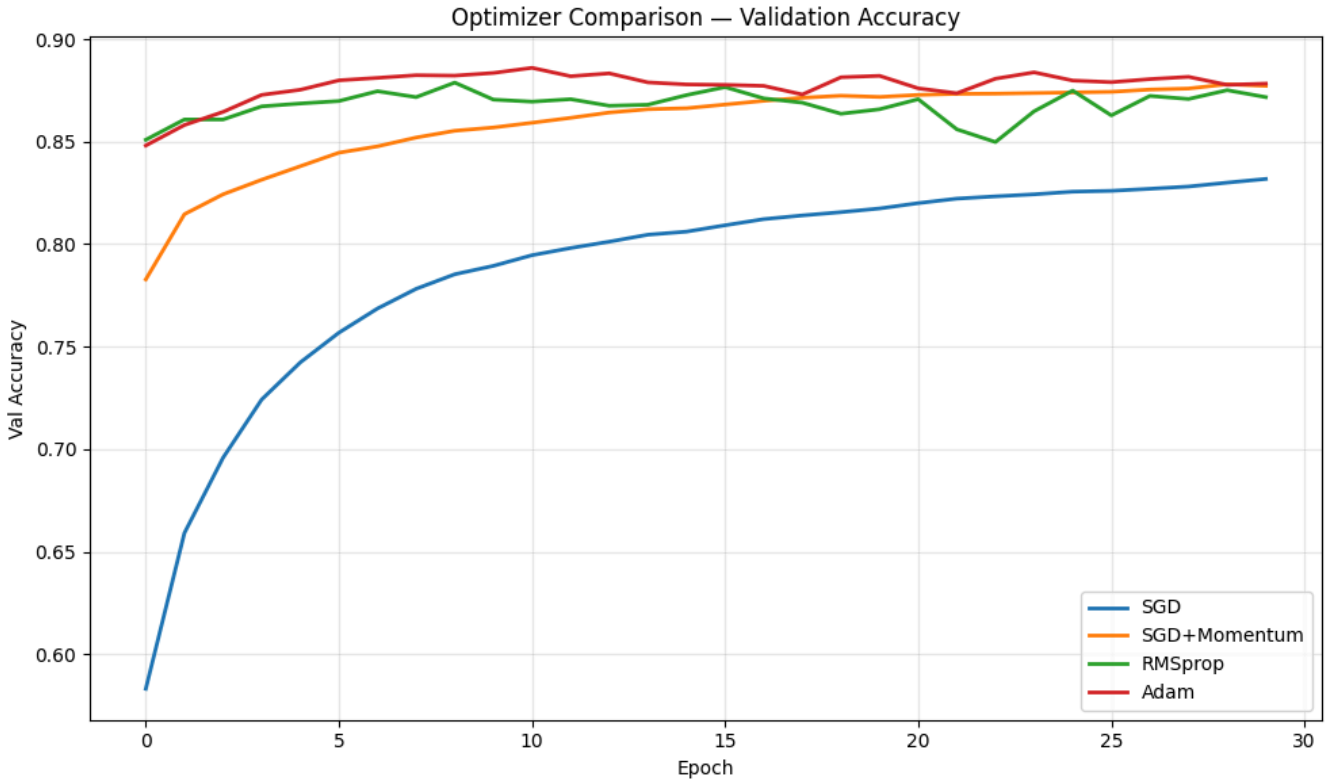

Figure 4b. Validation accuracy overlay for all four optimizers.

Optimizer	Train Acc	Val Acc	Val Loss	Best Val Ep
SGD	0.8374	0.8319	0.4859	30
SGD+Momentum	0.8924	0.8751	0.3520	30
RMSprop	0.9406	0.8782	0.5537	9
Adam ★	0.9506	0.8844	0.4671	9

2.5 Experiment D — Architecture

No clear relationship between parameter count and val accuracy was observed. D5 (308k params) achieved the best val accuracy (0.8863), while D2 (407k params) ranked near the bottom. D4 (deep-narrow, 63k params) nearly matched D2 on accuracy (0.8803 vs 0.8819) with 6× fewer parameters — supporting depth over width for feature extraction. D1 (25k params) was the only clear underfitting case.

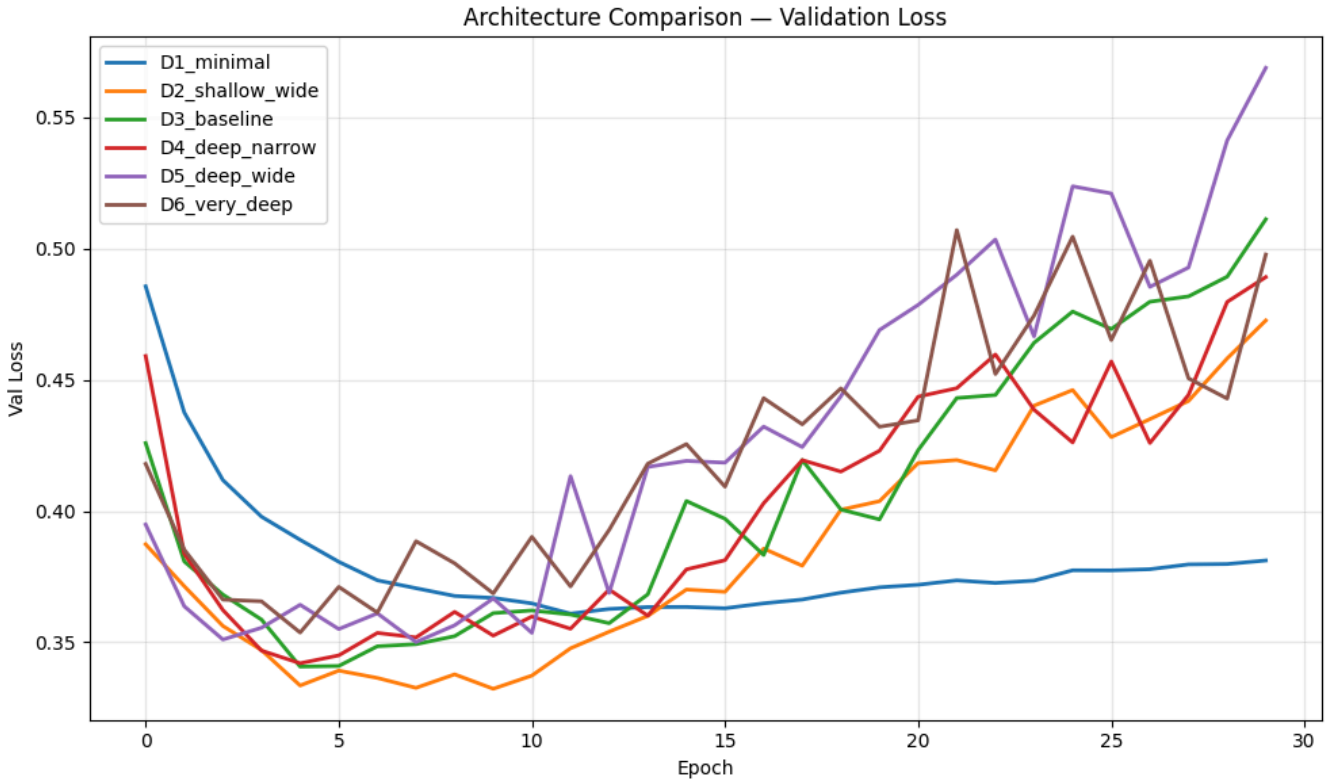

Figure 5a. Validation loss overlay for all six architectures.

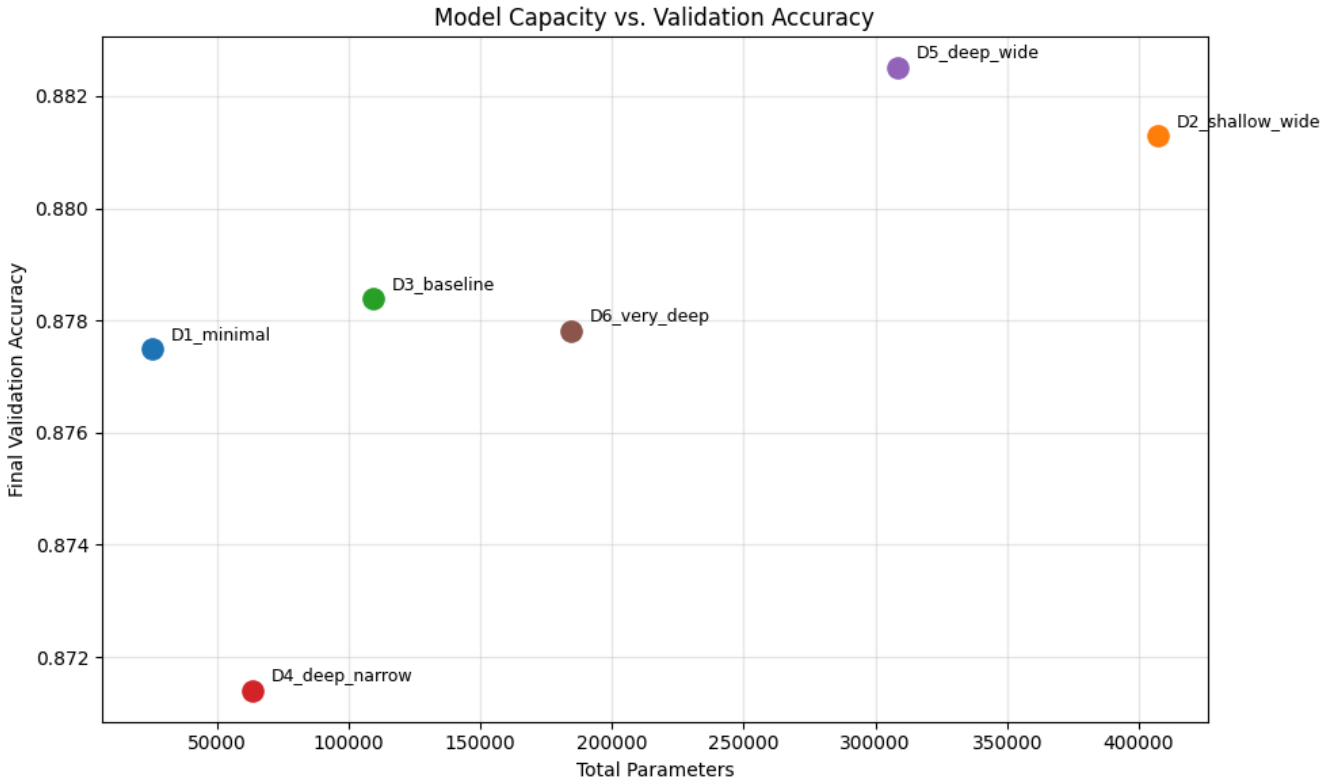

Figure 5b. Parameter count vs final validation accuracy. No monotonic relationship — D5 leads despite not having the most parameters.

3. Underfitting vs. Overfitting

Underfitting: D1 (Dense(32), 25k params) had the smallest train-val loss gap (0.0384) and lowest val accuracy (0.8762) — the model lacked capacity to learn the task. lr=0.00001 was a second case: both curves still descending at epoch 30 (best val epoch = 30), confirming the model was learning but too slowly to converge.

Overfitting: The baseline ANN shows the textbook pattern — val loss bottomed at epoch 6 (0.336) then rose to 0.474 by epoch 30, while training loss fell to 0.128. The CNN overfit even more dramatically: training accuracy reached 0.994 while val loss climbed from 0.244 (epoch 5) to 0.656 by epoch 30. D2 (shallow-wide) was the worst ANN overfitter, with the largest train-val gap (0.0816).

Loss-accuracy discrepancy: Validation accuracy curves appeared flat or improving while validation loss was actively rising across all experiments. Loss — which measures probability calibration — is the more sensitive overfitting signal; accuracy measures only discrete correctness and can remain stable as the model becomes miscalibrated.

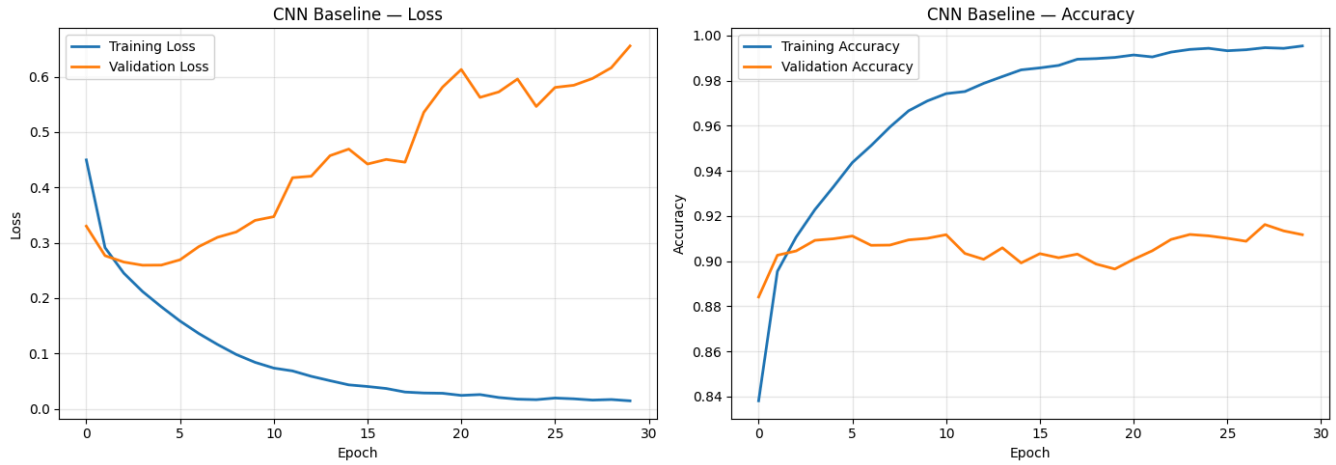

Figure 6. CNN training curves: training accuracy reaches 0.994 while val loss climbs from 0.244 (epoch 5) to 0.656 by epoch 30 — severe overfitting.

4. ANN vs. CNN

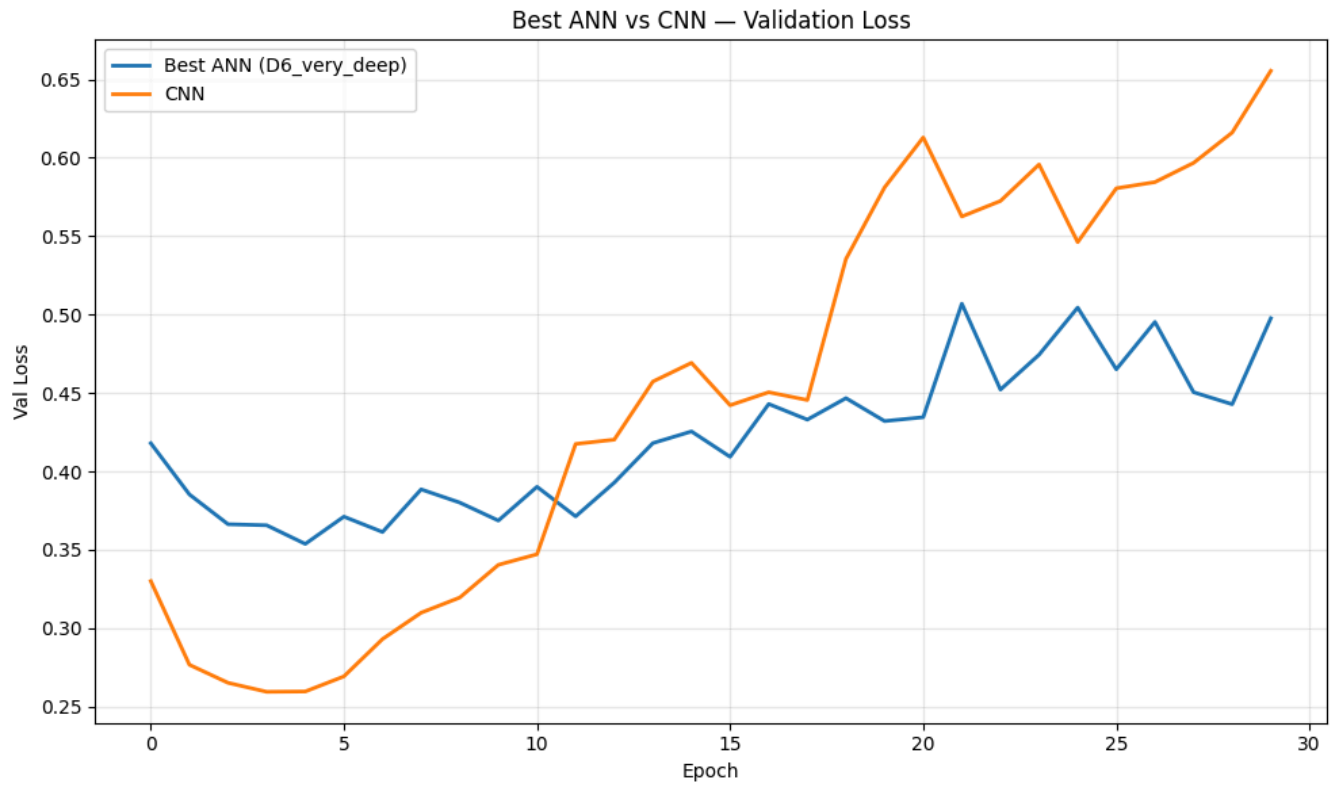

Figure 7. Validation loss and accuracy overlay: best ANN (D6 very-deep) vs CNN.

Model	Params	Train Acc	Val Acc	Best Val Loss	Best Val Ep
Baseline ANN	109,386	0.9530	0.8841	0.3363	6
CNN Baseline	~122k	0.9940	0.9091	0.2436	5

The CNN reached 0.909 val accuracy vs the ANN's 0.884 — a 2.5pp advantage with a similar parameter budget. This is explained by **inductive bias**: **local receptive fields** match the spatial structure of images (nearby pixels carry related information); **parameter sharing** applies the same filter across the entire image, giving **translational equivariance** — a shifted garment produces the same activations, shifted. An ANN flattens the image immediately, discarding all spatial relationships.

The CNN converged faster (best val loss epoch 5 vs 6) but overfit more severely afterward. For spatially-structured data, CNNs are the correct architectural prior. For tabular or sequence data without spatial locality, ANNs or attention-based architectures are more appropriate.

5. Final Model Rationale and Test Evaluation

Decision	Choice	Justification
Architecture	CNN	Best val acc (~0.91) vs best ANN (0.884)
Hidden layers	Conv32→Pool→Conv64→Pool→Dense128	Specified CNN; justified in Section 4
Learning rate	0.001	Lowest CNN val loss (0.244 at epoch 5)
Batch size	256	Stable gradients; low val loss (0.392)
Optimizer	Adam	Canonical for CNNs; fast adaptive convergence
Epochs	30 + EarlyStopping(patience=2)	Captures best-val epoch; prevents overfit

Test result: Test Loss = 0.2638 · Test Accuracy = 0.9102

Test accuracy (0.9102) matched the best validation accuracy (0.9122 at epoch 13), confirming the model generalised without overfitting to the validation set. Early stopping fired at epoch 15.

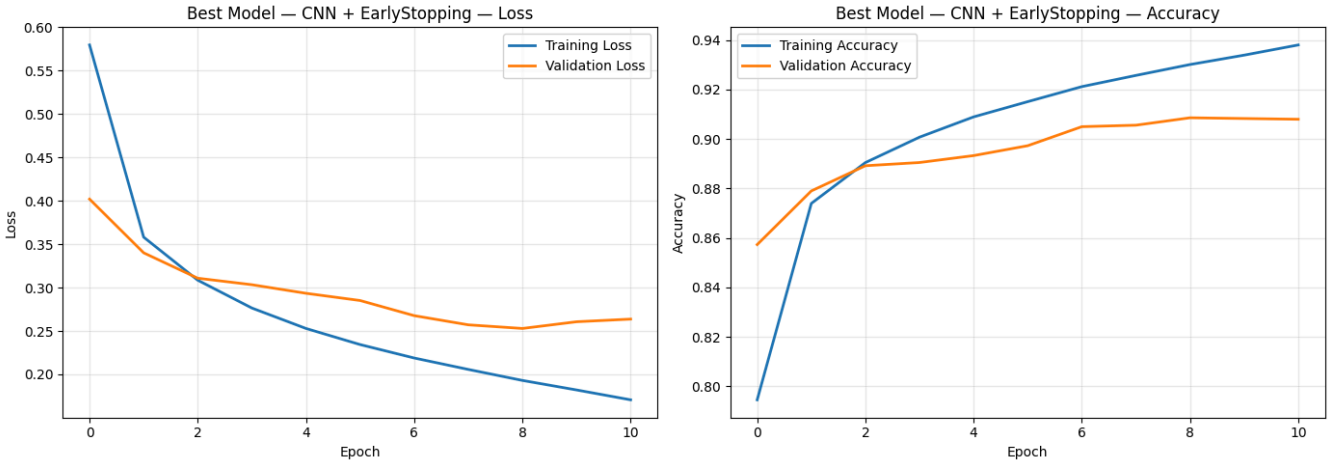

Figure 8. Best model training curves. Best val loss at epoch 13.

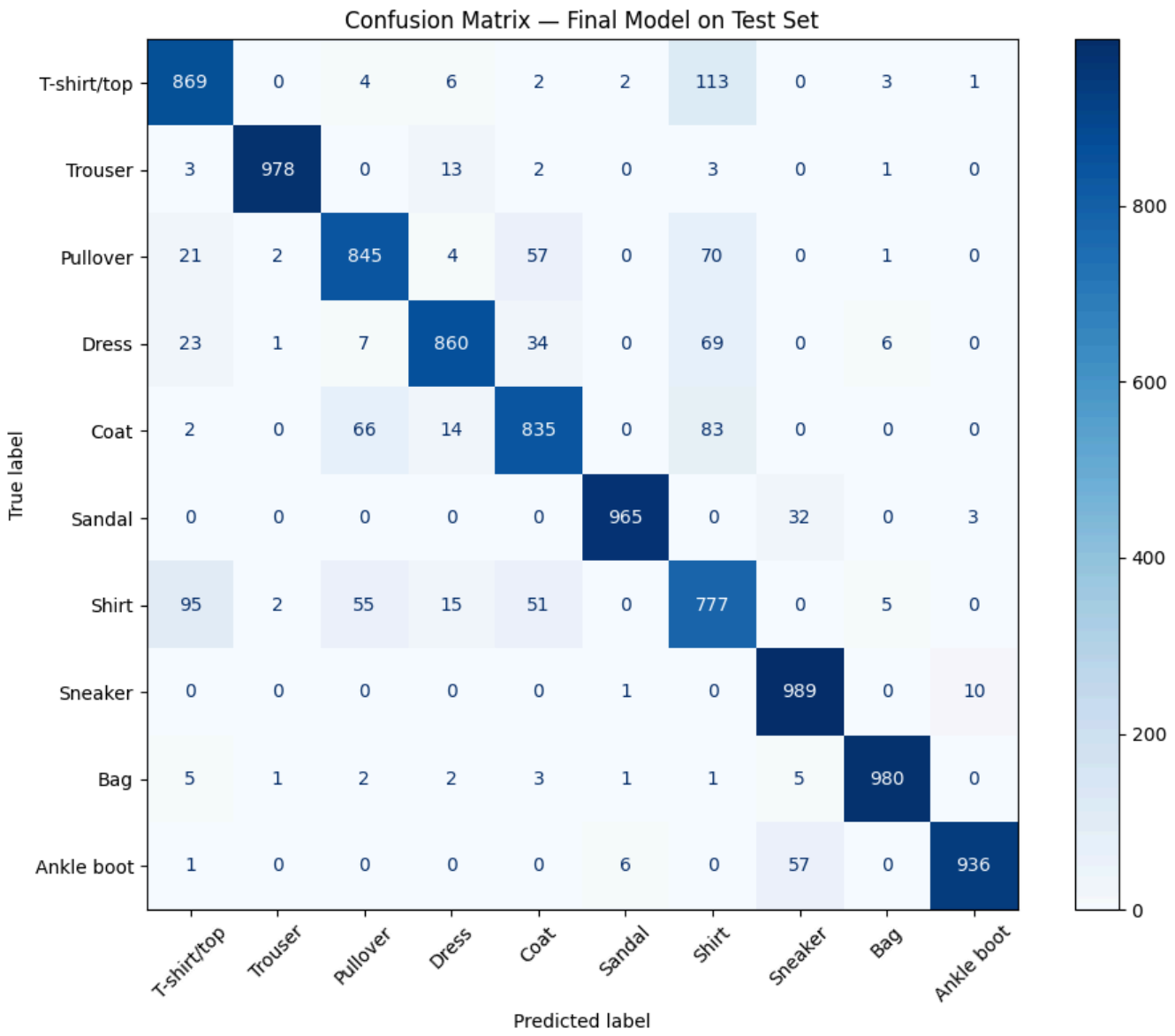

Figure 9. Confusion matrix (10,000 test samples). Most errors are intra-category: T-shirt ↔ Shirt (203 errors), Sneaker ↔ Ankle boot (69). Cleanest classes: Trouser, Bag, Sandal.

What to try next: Data augmentation (random flips + crops) is the highest-priority intervention — the 8.5pp train-val gap points to memorisation of fixed-orientation images. Early stopping at epoch 5 would recover val loss 0.244 vs the final 0.656 at zero cost. Dropout (0.3–0.5) after Dense(128) would provide complementary regularisation.

6. Voluntary Extension: Optimizer Deep Dive

Seven optimizers were tested on the CNN architecture (lr=0.001, bs=64, 30 epochs) to connect each optimizer's internal mechanics to its observed training behaviour. Note: Nadam did not complete training within the session; results cover six optimizers.

Optimizer	Mechanism	Val Acc	Val Loss	Best Ep
SGD	Pure gradient descent — no memory, no adaptation	0.8188	0.4844	30
SGD+Nesterov	Momentum + look-ahead gradient at anticipated next position	0.9017	0.2810	29
Adagrad	Per-param adaptive LR — divides by cumulative squared grads	0.8525	0.4228	30
RMSprop	Exponential moving avg of squared grads (fixes Adagrad)	0.9129	0.7623	5
Adam	RMSprop + momentum: adapts LR and smooths direction	0.9136	0.6410	4
AdamW	Adam + decoupled weight decay (genuine regularisation)	0.9075	0.5861	5

Individual training curves

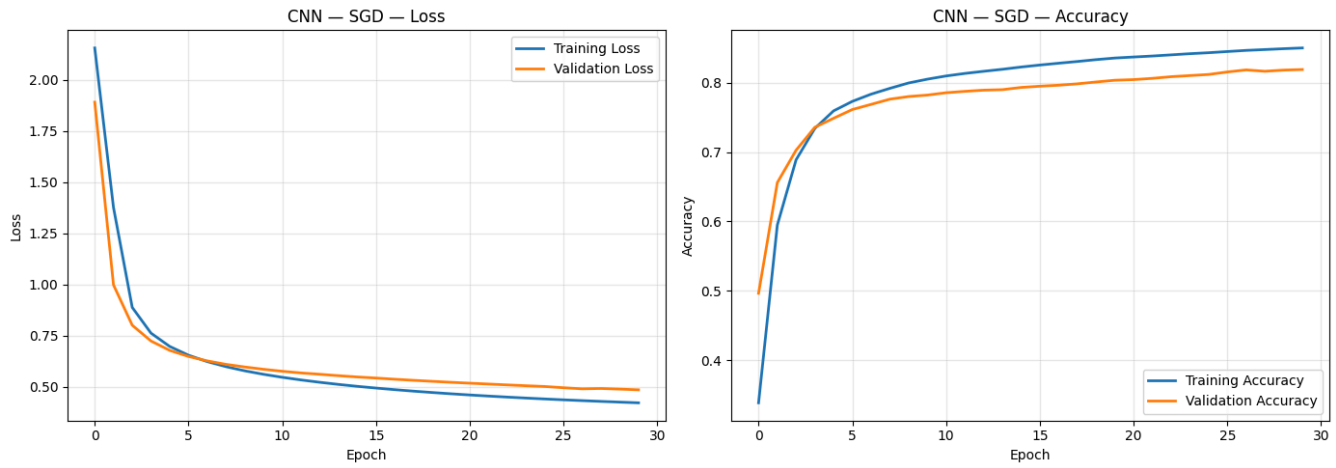

Figure 10a. SGD: slow, monotonic descent. Val loss still decreasing at epoch 30 — not converged. Lowest val accuracy (0.819) but zero overfitting.

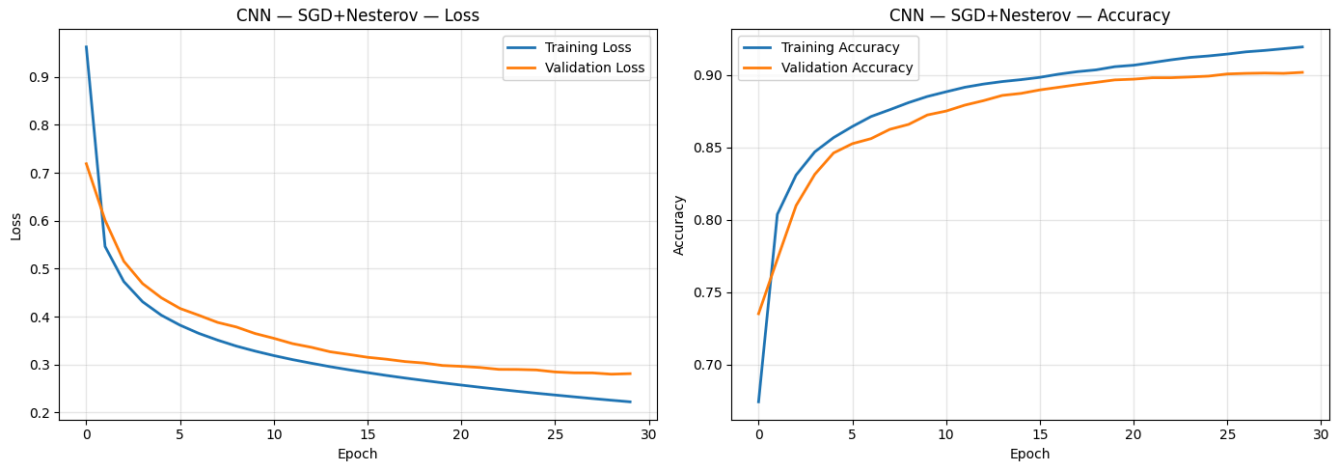

Figure 10b. SGD+Nesterov: look-ahead gradient accelerates convergence vs plain SGD. Val loss still descending at epoch 29 — nearly converged, val acc 0.902.

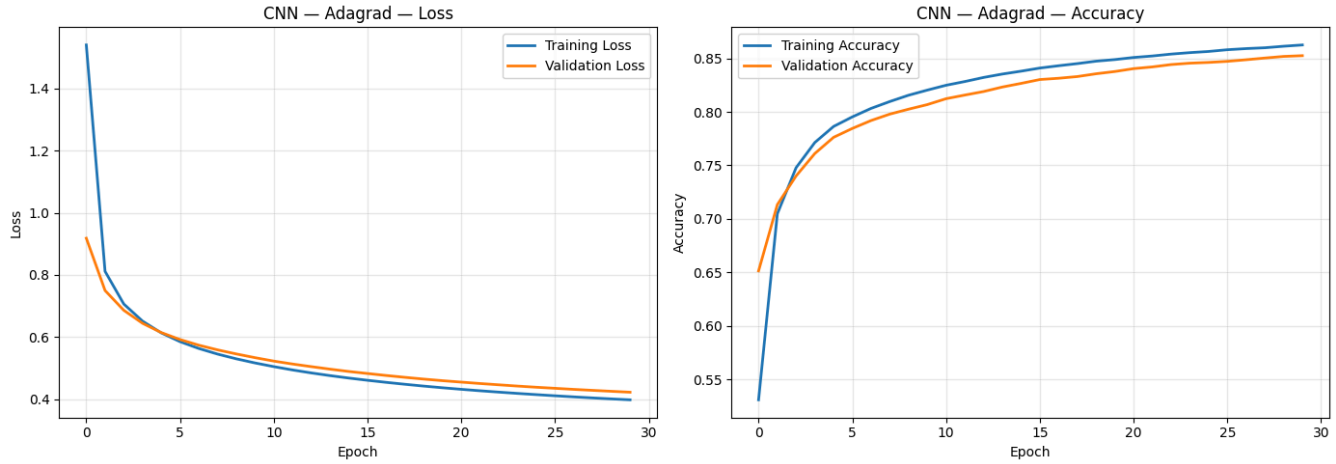

Figure 10c. Adagrad: fast early descent then hard plateau — cumulative squared-gradient denominator shrinks effective LR toward zero, preventing further learning.

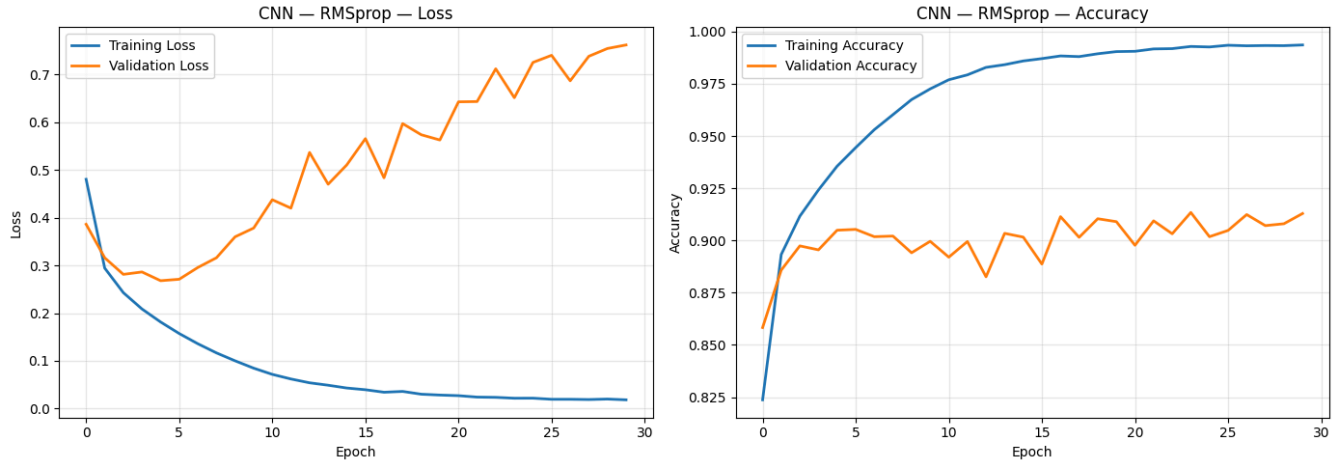

Figure 10d. RMSprop: fast convergence to best val loss at epoch 5, then severe divergence to 0.762 — lacks the first-moment term that stabilises Adam.

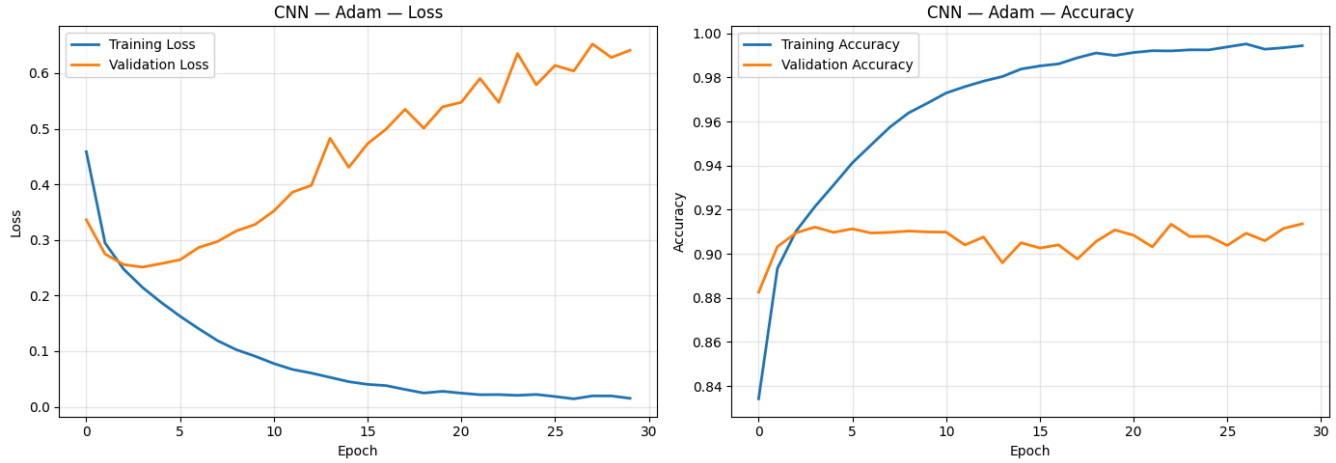

Figure 10e. Adam: fastest convergence, best val accuracy (0.914). Classic overfit divergence after best val epoch (4).

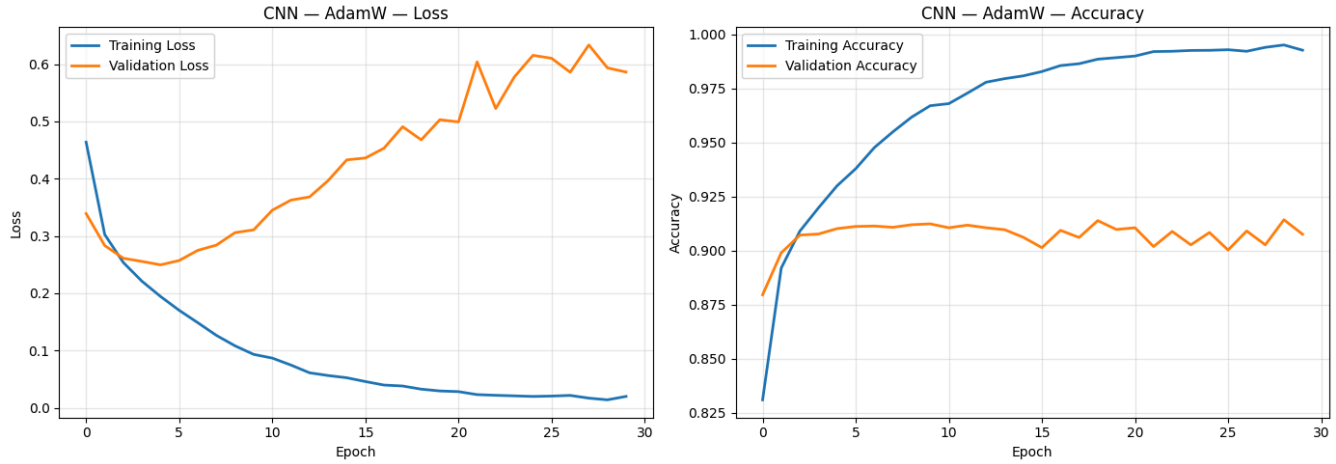

Figure 10f. AdamW: Adam speed with decoupled weight decay. Val loss diverges more slowly than Adam (0.586 vs 0.641 final) — genuine regularisation effect.

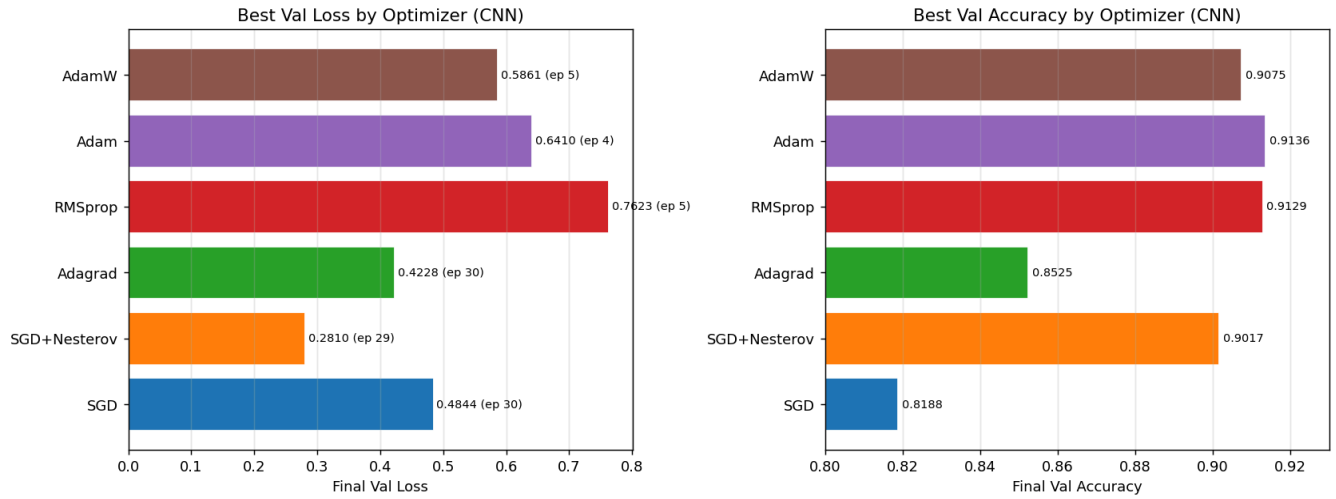

Figure 11. Summary: best val loss (with epoch) and final val accuracy by optimizer.

Mechanics vs. observed behaviour

SGD family (SGD → SGD+Nesterov): Both curves are smooth and still descending at epoch 30, confirming the slow-but-stable characteristic of non-adaptive methods. SGD+Nesterov's look-ahead gradient — computing the gradient at the anticipated post-momentum position — accelerates convergence without adding noise: val acc 0.902 vs 0.819 at epoch 30, tighter val loss curve. Neither overfit within 30 epochs, consistent with the literature showing SGD methods find flatter, more generalisable minima.

Adagrad: The characteristic plateau is clearly visible — early epochs show rapid descent (large initial LR on low-gradient parameters), then the curve flattens completely as the cumulative squared-gradient denominator grows without bound. The model effectively stops learning around epoch 10–15, ending at val acc 0.853.

RMSprop vs Adam: Both converge fast to their best val loss (epochs 5 and 4 respectively). After that, RMSprop diverges more steeply (0.762 vs 0.641 final val loss): without a first-moment term, RMSprop's updates are noisier in direction — it scales gradient magnitude well but doesn't smooth the gradient direction, leading to larger overshoots after reaching a good minimum.

AdamW vs Adam: Standard Adam applies L2 regularisation by adding the weight penalty to the gradient before adaptive scaling — the penalty gets divided by the gradient-variance denominator, weakening it for the most important parameters. AdamW decouples weight decay: it applies the shrinkage step directly to weights after the Adam update, independent of gradient scaling. The result is genuine regularisation: AdamW's val loss diverges more slowly (final 0.586 vs 0.641), with a smaller train-val gap — exactly what decoupled weight decay is designed to achieve.